VaultCore Banking System

Technical Report: Relational Database Architecture & SQL Suite

Target Engine:	PostgreSQL / Supabase	**Document Type:**	Technical Specification
Project Status:	Core Operational Suite	**Date:**	October 2026

1 System Overview & Architecture

The **VaultCore** database schema provides a robust enterprise banking foundation featuring core relational structures, explicit transactional boundaries, and operational audit trails. Designed for execution within PostgreSQL and Supabase environments, the schema handles user identity management, multi-branch operations, multi-account ledger balances, financial transfers with ACID safety, and integrated customer support ticketing.

2 Data Definition Language (DDL) — Schema Architecture

The relational model consists of 10 interconnected tables engineered with foreign key cascading and explicit integrity constraints.

```sql
-- 1. Branches Table
CREATE TABLE branches (
    branch_id INT PRIMARY KEY,
    branch_name VARCHAR(100) NOT NULL,
    district VARCHAR(100) NOT NULL,
    address VARCHAR(255) NOT NULL,
    IFSC_code VARCHAR(20) UNIQUE NOT NULL,
    phone VARCHAR(15) NOT NULL
);

-- 2. Users Table
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL,
    is_verified BOOLEAN DEFAULT FALSE
);

-- 3. User Profiles Table
CREATE TABLE user_profiles (
    profile_id INT PRIMARY KEY,
    user_id INT REFERENCES users(user_id) ON DELETE CASCADE,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    phone VARCHAR(15) NOT NULL,
    DOB DATE,
    gender VARCHAR(20),
    address VARCHAR(255)
);

```

```sql
-- 4. Customers Table
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    user_id INT REFERENCES users(user_id) ON DELETE CASCADE,
    branch_id INT REFERENCES branches(branch_id),
    customer_number VARCHAR(20) UNIQUE NOT NULL
);

-- 5. Account Types Table
CREATE TABLE account_types (
    type_id SERIAL PRIMARY KEY,
    type_name VARCHAR(30) UNIQUE NOT NULL,
    minimum_balance NUMERIC(15,2) DEFAULT 0,
    interest_rate NUMERIC(5,2) DEFAULT 0
);

-- 6. Accounts Table
CREATE TABLE accounts (
    account_id INT PRIMARY KEY,
    customer_id INT REFERENCES customers(customer_id) ON DELETE CASCADE,
    type_id INT REFERENCES account_types(type_id),
    branch_id INT REFERENCES branches(branch_id),
    account_number VARCHAR(20) UNIQUE NOT NULL,
    balance NUMERIC(15,2) NOT NULL DEFAULT 0.00,
    status VARCHAR(20) DEFAULT 'Active'
);

-- 7. Transactions Table
CREATE TABLE transactions (
    transaction_id INT PRIMARY KEY,
    account_id INT REFERENCES accounts(account_id) ON DELETE CASCADE,
    transaction_type VARCHAR(50) NOT NULL,
    amount NUMERIC(15,2) NOT NULL,
    receiver_account VARCHAR(20),
    description TEXT,
    transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 8. Transfers Table
CREATE TABLE transfers (
    transfer_id SERIAL PRIMARY KEY,
    sender_account_id INT REFERENCES accounts(account_id),
    receiver_account_id INT REFERENCES accounts(account_id),
    amount NUMERIC(15,2) NOT NULL,
    transfer_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(20) DEFAULT 'Completed'
);

-- 9. Transaction Logs Table
CREATE TABLE transaction_logs (
    log_id SERIAL PRIMARY KEY,
    transaction_id INT REFERENCES transactions(transaction_id) ON DELETE
       CASCADE,
    performed_by INT REFERENCES users(user_id),
    action VARCHAR(100),
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    remarks TEXT
);

-- 10. Support Tickets Table
```

```sql
CREATE TABLE support_tickets (
    ticket_id SERIAL PRIMARY KEY,
    customer_id INT REFERENCES customers(customer_id) ON DELETE CASCADE,
    subject VARCHAR(150) NOT NULL,
    issue TEXT NOT NULL,
    status VARCHAR(50) DEFAULT 'Open',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Listing 1: DDL Definition for Core Entities

3 Data Insertion (Seeding Scripts)

The following seed scripts populate initial operational entities across branches, identities, products, accounts, and ticketing modules.

```sql
-- Inserting Branches
INSERT INTO branches (branch_id, branch_name, district, address, IFSC_code, phone) VALUES
(1, 'VaultCore Trivandrum', 'Thiruvananthapuram', 'MG Road, Thiruvananthapuram', 'VCBK0001001', '9876543210'),
(2, 'VaultCore Kochi', 'Ernakulam', 'Marine Drive, Kochi', 'VCBK0001002', '9876543211'),
(3, 'VaultCore Kozhikode', 'Kozhikode', 'Mavoor Road, Kozhikode', 'VCBK0001003', '9876543212');

-- Inserting Users
INSERT INTO users (user_id, email, password_hash, role, is_verified) VALUES
(1, 'admin@vaultcore.com', '$2b$12$AdminHash123', 'Admin', TRUE),
(2, 'neha@gmail.com', '$2b$12$UserHash001', 'User', TRUE),
(3, 'karthika@gmail.com', '$2b$12$UserHash002', 'User', TRUE);

-- Inserting Account Types
INSERT INTO account_types (type_id, type_name, minimum_balance, interest_rate) VALUES
(1, 'Savings', 500.00, 3.50),
(2, 'Current', 1000.00, 0.00);

-- Inserting Accounts
INSERT INTO accounts (account_id, customer_id, type_id, branch_id, account_number, balance, status) VALUES
(1, 1, 1, 1, '100100000001', 25000.00, 'Active'),
(2, 2, 1, 1, '100100000002', 18500.50, 'Active');

-- Inserting Support Tickets
INSERT INTO support_tickets (ticket_id, customer_id, subject, issue, status) VALUES
(1, 1, 'Password Reset', 'Unable to log in after password change.', 'Open'),
(2, 2, 'Balance Inquiry', 'Balance not updated immediately.', 'Closed'),
(3, 3, 'Transaction Failed', 'Transfer failed on first attempt.', 'Open'),
(4, 4, 'Account Statement', 'Need account statement for last month.', 'Resolved');
```

Listing 2: Sample Data Insertion Scripts

4 Operational & Application Queries

4.1 Customer & Account Retrieval Queries

```sql
-- Fetching basic customer list
SELECT customer_id, customer_number FROM customers;

-- Checking account balance for a specific account number
SELECT account_number, balance
FROM accounts
WHERE account_number = '100100000001';

-- Deactivating an account
UPDATE accounts
SET status = 'INACTIVE'
WHERE account_id = 1;

-- Fetching complete history for a specific account ordered by recency
SELECT transaction_id, account_id, transaction_type, amount, transaction_date,
    description
FROM transactions
WHERE account_id = 1
ORDER BY transaction_date DESC;
```

Listing 3: Retrieving Customer & Account Details

4.2 Cross-Table Reporting & Analytical Joins

```sql
-- Joining Customers and Accounts
SELECT c.customer_number, u.email, a.account_number, a.balance, a.status
FROM customers c
JOIN users u ON c.user_id = u.user_id
JOIN accounts a ON c.customer_id = a.customer_id;

-- Branch-Wise Accounts Report
SELECT b.branch_name, a.account_number, a.balance
FROM branches b
JOIN accounts a ON b.branch_id = a.branch_id;

-- Counting total accounts in the system
SELECT COUNT(*) AS total_accounts FROM accounts;

-- Calculating total funds managed by the bank
SELECT SUM(balance) AS total_bank_balance FROM accounts;
```

Listing 4: Reporting and Analytics Queries

4.3 ACID Transaction Control (Fund Transfers)

```sql
-- Fund transfer safety block using explicit transaction control
BEGIN;

-- 1. Deduct from sender
UPDATE accounts
SET balance = balance - 2500.00
WHERE account_id = 1;

-- 2. Add to receiver
UPDATE accounts
```

```sql
SET balance = balance + 2500.00
WHERE account_id = 2;

-- 3. Log the transfer record
INSERT INTO transfers (sender_account_id, receiver_account_id, amount, status)
VALUES (1, 2, 2500.00, 'Completed');

COMMIT;
```

Listing 5: Atomic Fund Transfer Block

4.4 Support Ticket Module Queries

```sql
-- 1. Fetching all open support tickets for administrative review
SELECT ticket_id, customer_id, subject, issue, created_at
FROM support_tickets
WHERE status = 'Open';

-- 2. Updating support ticket status
UPDATE support_tickets
SET status = 'Resolved'
WHERE ticket_id = 1;
```

Listing 6: Support Module Queries